Session 20 (AIML) - Assignment Questions

Name**: Vaishnavi Praveen Rathi**
College**: Zeal Polytechnic, Narhe, Pune**
Branch**: Artificial Intelligence and Machine Learning (AIML)**
Domain**: AIML (Artificial Intelligence and Machine Learning)**

Github Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session20/Assignment20.ipynb

Dataset Used: – Kaggle - Ford Car Dataset

Q1. Preparing Features

Output Screenshots:

```
Shape of X: (17966, 8)
Shape of Y: (17966,)

Independent Features:
['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg', 'engineSize']

Dependent Feature:
price
```

Program Code:

[2]:
```
'''
Q1. (Preparing Features)

Load your cleaned Ford Car Dataset.
Separate the dataset into Independent Features (X) and Dependent Feature (Y) (price).
Drop the target column from X.
Print the shape of X and Y.
'''

import pandas as pd

# Load the cleaned dataset
df = pd.read_csv("ford_car_dataset.csv")

# Separate independent and dependent features
X = df.drop("price", axis=1)
Y = df["price"]

# Display shapes
print("Shape of X:", X.shape)
print("Shape of Y:", Y.shape)

# Display feature names
print("\nIndependent Features:")
print(X.columns.tolist())

print("\nDependent Feature:")
print(Y.name)
```

Q2. One-Hot Encoding

Output Screenshot:

```
Categorical Columns:
['model', 'transmission', 'fuelType']

First 5 Rows After One-Hot Encoding:
   year  mileage  tax  mpg  engineSize  model_C-MAX  model_EcoSport  \
0  2017    15944  150   57           1            0               0
1  2018     9083  150   57           1            0               0
2  2017    12456  150   57           1            0               0
3  2019    10460  145   40           1            0               0
4  2019     1482  145   48           1            0               0

   model_Edge  model_Escort  model_Fiesta  ...  model_Streetka  \
0           0             0             1  ...               0
1           0             0             0  ...               0
2           0             0             0  ...               0
3           0             0             1  ...               0
4           0             0             1  ...               0

   model_Tourneo Connect  model_Tourneo Custom  model_Transit Tourneo  \
0                      0                     0                      0
1                      0                     0                      0
2                      0                     0                      0
3                      0                     0                      0
4                      0                     0                      0

   transmission_Manual  transmission_Semi-Auto  fuelType_Electric  \
0                    0                       0                  0
1                    1                       0                  0
2                    1                       0                  0
3                    1                       0                  0
4                    0                       0                  0

   fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0                0              0                1
1                0              0                1
2                0              0                1
3                0              0                1
4                0              0                1

[5 rows x 33 columns]
```

Program Code:

[3]:
```
'''
Q2. (One-Hot Encoding)

Identify all categorical columns in X.
Perform One-Hot Encoding using pd.get_dummies().
Convert the encoded data to integer type (astype(int)).
Show the first 5 rows after encoding.
'''

# Identify categorical columns
categorical_columns = X.select_dtypes(include="object").columns

print("Categorical Columns:")
print(categorical_columns.tolist())

# Apply One-Hot Encoding
X_encoded = pd.get_dummies(
    X,
    columns=categorical_columns,
    drop_first=True
)

# Convert boolean columns to integer
X_encoded = X_encoded.astype(int)

# Display first 5 rows
print("\nFirst 5 Rows After One-Hot Encoding:")
print(X_encoded.head())
```

Q3.Feature Scaling

Output Screenshot:

```
First 5 Rows of Scaled Features:
       year   mileage       tax       mpg  engineSize  model_C-MAX  \
0  0.065128 -0.380998  0.591358 -0.042122   -0.447984            0
1  0.552866 -0.733359  0.591358 -0.042122   -0.447984            0
2  0.065128 -0.560132  0.591358 -0.042122   -0.447984            0
3  1.040605 -0.662640  0.510727 -1.721198   -0.447984            0
4  1.040605 -1.123724  0.510727 -0.931045   -0.447984            0

   model_EcoSport  model_Edge  model_Escort  model_Fiesta  ...  \
0               0           0             0             1  ...
1               0           0             0             0  ...
2               0           0             0             0  ...
3               0           0             0             1  ...
4               0           0             0             1  ...

   model_Streetka  model_Tourneo Connect  model_Tourneo Custom  \
0               0                      0                     0
1               0                      0                     0
2               0                      0                     0
3               0                      0                     0
4               0                      0                     0

   model_Transit Tourneo  transmission_Manual  transmission_Semi-Auto  \
0                      0                    0                       0
1                      0                    1                       0
2                      0                    1                       0
3                      0                    1                       0
4                      0                    0                       0
```

```
   fuelType_Electric  fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0                  0                0               0                1
1                  0                0               0                1
2                  0                0               0                1
3                  0                0               0                1
4                  0                0               0                1

[5 rows x 33 columns]
```

Program Code:

[4]:
```
'''
Q3. (Feature Scaling)

Apply Standard Scaling on the numerical columns (year, mileage, tax, mpg,
engineSize).
Use StandardScaler from sklearn.preprocessing.
Show the first 5 rows of the scaled features.
'''

from sklearn.preprocessing import StandardScaler

# Numerical columns to scale
numeric_columns = ["year", "mileage", "tax", "mpg", "engineSize"]

# Create scaler
scaler = StandardScaler()

# Scale numerical features
X_encoded[numeric_columns] = scaler.fit_transform(X_encoded[numeric_columns])

# Display first 5 rows
print("First 5 Rows of Scaled Features:")
print(X_encoded.head())
```

Q4. Train-Test Split

Output Screenshot:

```
X_train Shape: (12037, 33)
X_test Shape : (5929, 33)
y_train Shape: (12037,)
y_test Shape : (5929,)
```

Program Code:

[5]:
```
'''
Q4. (Train-Test Split)

Split the preprocessed data into training and testing sets using train_test_split.
Use test_size=0.33 and random_state=42.
Print the shape of X_train, X_test, y_train, and y_test.
'''

from sklearn.model_selection import train_test_split

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    Y,
    test_size=0.33,
    random_state=42
)

# Display shapes
print("X_train Shape:", X_train.shape)
print("X_test Shape :", X_test.shape)
print("y_train Shape:", y_train.shape)
print("y_test Shape :", y_test.shape)
```

Q5. Building Linear Regression Model

Output Screenshot:

```
Intercept:
11609.497928349992

Model Coefficients:
                          Feature   Coefficient
0                            year  2.038189e+03
1                         mileage -1.318488e+03
2                             tax -2.060502e+01
3                             mpg -9.889718e+02
4                      engineSize  8.796971e+02
5                     model_C-MAX  9.713849e+02
6                  model_EcoSport  1.288293e+03
7                      model_Edge  8.451730e+03
8                    model_Escort  1.390368e+04
9                    model_Fiesta  9.262776e+02
10                    model_Focus  3.016096e+03
11                   model_Fusion  3.491117e+03
12                   model_Galaxy  5.689973e+03
13              model_Grand C-MAX  1.237606e+03
14  model_Grand Tourneo Connect  4.014553e+03
15                       model_KA -4.406591e+02
16                     model_Ka+ -2.910095e+03
17                     model_Kuga  3.226758e+03
18                   model_Mondeo  2.418454e+03
19                  model_Mustang  1.302115e+04
20                     model_Puma  7.218829e+03
21                   model_Ranger  1.682565e-11
22                    model_S-MAX  5.533757e+03
23                  model_Streetka -9.094947e-13
24           model_Tourneo Connect  3.425333e+03
25            model_Tourneo Custom  5.550494e+02
26          model_Transit Tourneo  8.695450e+02
27            transmission_Manual -5.091894e+02
28       transmission_Semi-Auto -2.480119e+02
29            fuelType_Electric -4.547474e-13
30              fuelType_Hybrid  8.645643e+03
31               fuelType_Other  6.599495e+02
32              fuelType_Petrol -1.300651e+03
```

Program Code:

[6]:
```
'''
Q5. (Building Linear Regression Model)

Train a Linear Regression model using sklearn.
Fit the model on the training data (X_train, y_train).

Print the intercept and coefficients of the model.
'''

from sklearn.linear_model import LinearRegression

# Create the model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Print intercept
print("Intercept:")
print(model.intercept_)

# Print coefficients
coefficients = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

print("\nModel Coefficients:")
print(coefficients)
```

Q6.Making Predictions

Output Screenshot:

```
First 10 Predicted Values:
[ 6886.43611974  9329.53758889  9419.36269037  4494.44391177
  2429.15179332 16980.39203353 28490.75684787 12484.09019267
 11373.10359113 17275.9738973 ]

Actual vs Predicted Prices:
   Actual Price  Predicted Price
0          6995      6886.436120
1          8999      9329.537589
2          7998      9419.362690
3          5491      4494.443912
4          3790      2429.151793
5         18200     16980.392034
6         22998     28490.756848
7         11000     12484.090193
8          7600     11373.103591
9         14985     17275.973897
```

Program Code:

```
[7]: '''
Q6. (Making Predictions)

Use the trained model to make predictions on the test data.
Store predictions in y_pred.
Print the first 10 predicted values and compare them with the first 10 actual
values from y_test.
'''

# Make predictions
y_pred = model.predict(X_test)

# Display first 10 predicted values
print("First 10 Predicted Values:")
print(y_pred[:10])

# Compare actual vs predicted values
comparison = pd.DataFrame({
    "Actual Price": y_test.iloc[:10].values,
    "Predicted Price": y_pred[:10]
})

print("\nActual vs Predicted Prices:")
print(comparison)
```

Q7. Model Evaluation using R2 Score

Output Screenshot:

```
R² Score: 0.8397
Model Accuracy (%): 83.97 %

Performance: Very good model performance.
```

Program Code:

```
[11]: '''
Q7. (Model Evaluation using R2 Score)

Calculate the R2 score using r2_score(y_test, y_pred).
Interpret the R2 value in your own words.
Comment on how well the model is performing.
'''

from sklearn.metrics import r2_score

# Calculate R² Score
r2 = r2_score(y_test, y_pred)

# Convert to percentage
accuracy = r2 * 100

print("R² Score:", round(r2, 4))
print("Model Accuracy (%):", round(accuracy, 2), "%")

# Performance message
if r2 >= 0.90:
    print("\nPerformance: Excellent model performance.")
elif r2 >= 0.75:
    print("\nPerformance: Very good model performance.")
elif r2 >= 0.50:
    print("\nPerformance: Good model performance.")
else:
    print("\nPerformance: The model needs improvement.")
```

```
"""
Interpretation:

1. The R² score represents the proportion of variation in house prices
   explained by the independent features.

2. A higher R² score (closer to 1) indicates that the model fits the data well.

3. The model accuracy percentage shows how much of the variation in the
   target variable is explained by the model.

4. If the R² score is low, additional feature engineering, data cleaning,
   or a different regression algorithm may improve the model's performance.
"""
```

Q8.Saving the Model

Output Screenshot:

```
Model and preprocessing objects saved successfully!
Files Created:
1. LR_ford_car.pkl
2. scaler.pkl
3. columns.pkl
```

Program Code:

[10]:
```
'''
Q8. (Saving the Model)

Save your trained model and preprocessing objects using joblib:
The Linear Regression model → "LR_ford_car.pkl"
The StandardScaler → "scaler.pkl"
The list of columns → "columns.pkl"
'''

import joblib

# Save the trained Linear Regression model
joblib.dump(model, "LR_ford_car.pkl")

# Save the StandardScaler
joblib.dump(scaler, "scaler.pkl")

# Save the feature column names
joblib.dump(X_encoded.columns.tolist(), "columns.pkl")

print("Model and preprocessing objects saved successfully!")
print("Files Created:")
print("1. LR_ford_car.pkl")
print("2. scaler.pkl")
print("3. columns.pkl")
```

Optional Task

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session20/Session20%20OptionalTask/Assignment20%20OptionalTask.ipynb

Dataset Used: Kaggle - https://www.kaggle.com/datasets/shree1992/housedata

Program Output:

1. Import Libraries

```
[38]: import pandas as pd
      import numpy as np
      import matplotlib.pyplot as plt
      import seaborn as sns
      import joblib

      from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
      from sklearn.linear_model import LinearRegression
      from sklearn.metrics import r2_score
```

2. Load the Dataset

```
[39]: # Load dataset
      df = pd.read_csv("House_Price_Prediction.csv")

      # Display first 5 rows
      df.head()
```

[39]:

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	yr_built	yr_r
0	2014-05-02 00:00:00	313000.0	3.0	1.50	1340	7912	1.5	0	0	3	1340	0	1955	
1	2014-05-02 00:00:00	2384000.0	5.0	2.50	3650	9050	2.0	0	4	5	3370	280	1921	
2	2014-05-02 00:00:00	342000.0	3.0	2.00	1930	11947	1.0	0	0	4	1930	0	1966	
3	2014-05-02 00:00:00	420000.0	3.0	2.25	2000	8030	1.0	0	0	4	1000	1000	1963	
4	2014-05-02 00:00:00	550000.0	4.0	2.50	1940	10500	1.0	0	0	4	1140	800	1976	

3. Explore the Dataset

[59]:
```
print("Shape of Dataset:", df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)
```

```
Shape of Dataset: (4551, 18)

Column Names:
Index(['price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors',
       'waterfront', 'view', 'condition', 'sqft_above', 'sqft_basement',
       'yr_built', 'yr_renovated', 'city', 'statezip', 'country', 'year',
       'month'],
      dtype='object')
```

```
Data Types:
price            float64
bedrooms         float64
bathrooms        float64
sqft_living        int64
sqft_lot           int64
floors           float64
waterfront         int64
view               int64
condition          int64
sqft_above         int64
sqft_basement      int64
yr_built           int64
yr_renovated       int64
city              object
statezip          object
country           object
year               int32
month              int32
dtype: object
```

4. Check Missing Values and Duplicate Records

```
[41]: # Missing Values
      print("Missing Values:")
      print(df.isnull().sum())

      # Duplicate Rows
      print("\nDuplicate Rows:", df.duplicated().sum())
```

```
Missing Values:
date             0
price            0
bedrooms         0
bathrooms        0
sqft_living      0
sqft_lot         0
floors           0
waterfront       0
view             0
condition        0
sqft_above       0
sqft_basement    0
yr_built         0
yr_renovated     0
street           0
city             0
statezip         0
country          0
dtype: int64

Duplicate Rows: 0
```

5. Clean the Dataset

```
[42]: # Remove missing values
      df.dropna(inplace=True)

      # Remove duplicate rows
      df.drop_duplicates(inplace=True)

      print("New Shape:", df.shape)
```

```
New Shape: (4600, 18)
```

6. Generate Statistical Summary

```
[43]: print(df.describe())

      print("\nPrice Statistics")
      print("Minimum :", df["price"].min())
      print("Maximum :", df["price"].max())
      print("Mean    :", df["price"].mean())
      print("Median  :", df["price"].median())
```

```
              price     bedrooms    bathrooms   sqft_living      sqft_lot  \
count  4.600000e+03  4600.000000  4600.000000   4600.000000  4.600000e+03
mean   5.519630e+05     3.400870     2.160815   2139.346957  1.485252e+04
std    5.638347e+05     0.908848     0.783781    963.206916  3.588444e+04
min    0.000000e+00     0.000000     0.000000    370.000000  6.380000e+02
25%    3.228750e+05     3.000000     1.750000   1460.000000  5.000750e+03
50%    4.609435e+05     3.000000     2.250000   1980.000000  7.683000e+03
75%    6.549625e+05     4.000000     2.500000   2620.000000  1.100125e+04
max    2.659000e+07     9.000000     8.000000  13540.000000  1.074218e+06

             floors   waterfront         view    condition   sqft_above  \
count  4600.000000  4600.000000  4600.000000  4600.000000  4600.000000
mean      1.512065     0.007174     0.240652     3.451739  1827.265435
std       0.538288     0.084404     0.778405     0.677230   862.168977
min       1.000000     0.000000     0.000000     1.000000   370.000000
25%       1.000000     0.000000     0.000000     3.000000  1190.000000
50%       1.500000     0.000000     0.000000     3.000000  1590.000000
75%       2.000000     0.000000     0.000000     4.000000  2300.000000
max       3.500000     1.000000     4.000000     5.000000  9410.000000

       sqft_basement     yr_built  yr_renovated
count    4600.000000  4600.000000   4600.000000
mean      312.081522  1970.786304    808.608261
std       464.137228    29.731848    979.414536
min         0.000000  1900.000000      0.000000
25%         0.000000  1951.000000      0.000000
50%         0.000000  1976.000000      0.000000
75%       610.000000  1997.000000   1999.000000
max      4820.000000  2014.000000   2014.000000

Price Statistics
Minimum : 0.0
Maximum : 26590000.0
Mean    : 551962.9884732141
Median  : 460943.46153850004
```

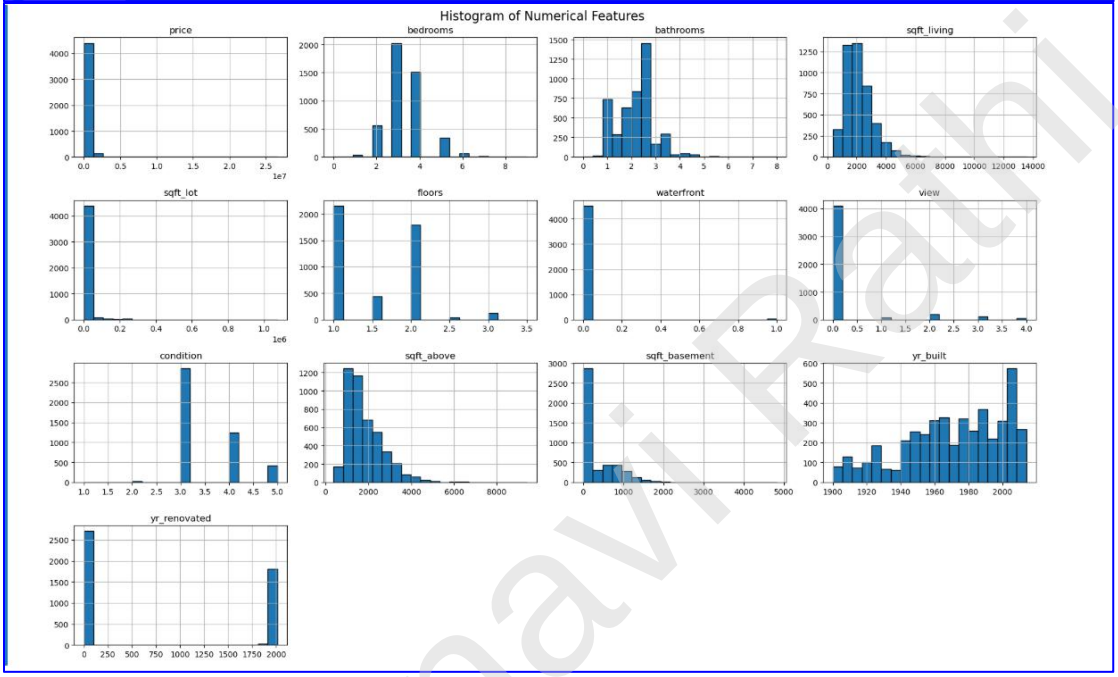
7. Remove Invalid Price Values

```
[45]: # Remove houses with invalid price

      df = df[df["price"] > 0]

      print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (4551, 18)
```

8. Histogram Analysis

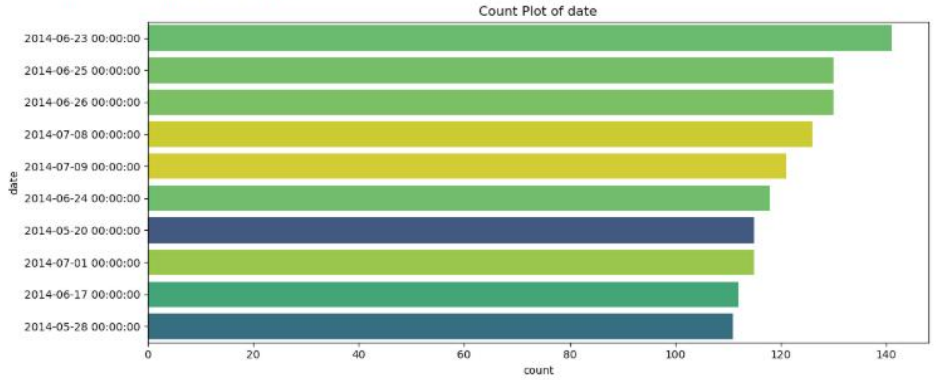
[46]:
```
# Select numerical columns
numeric_columns = df.select_dtypes(include=["int64", "float64"]).columns

# Plot histograms
df[numeric_columns].hist(figsize=(18, 12), bins=20, edgecolor="black")

plt.suptitle("Histogram of Numerical Features", fontsize=16)
plt.tight_layout()
plt.show()
```

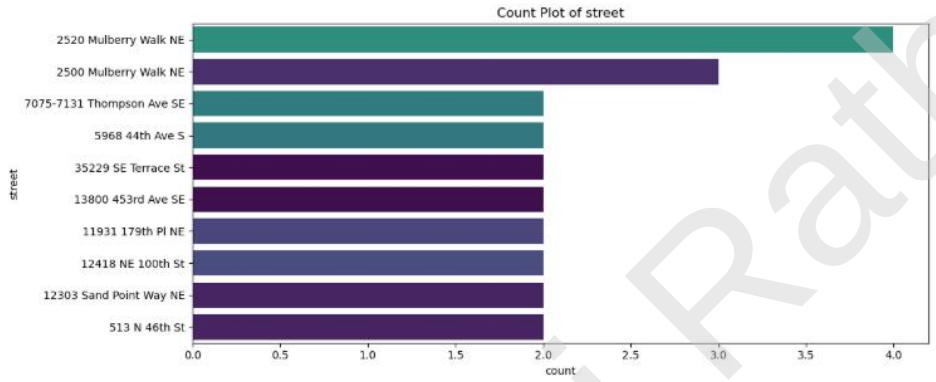


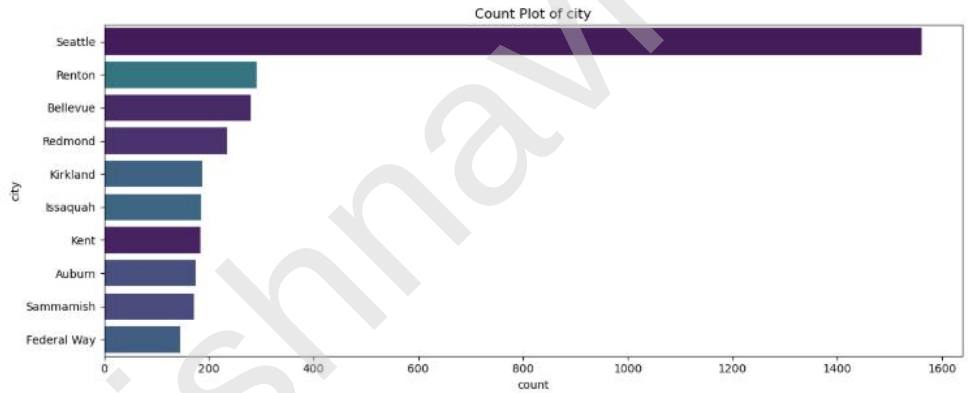
9. Count Plot Analysis

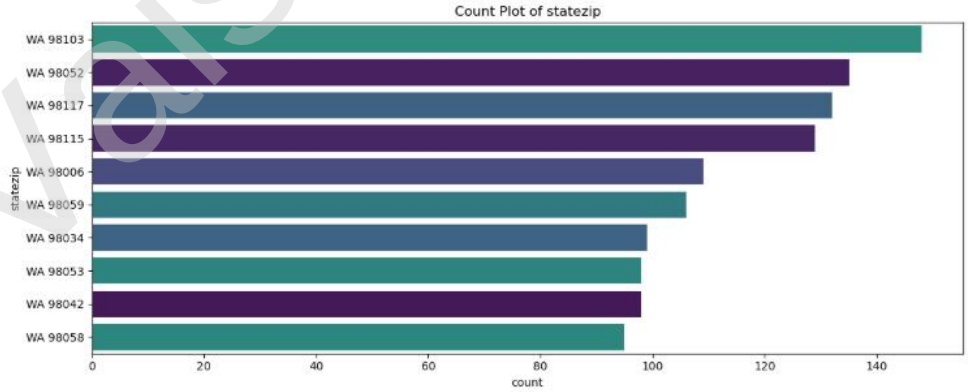
[47]:
```
# Identify categorical columns
categorical_columns = df.select_dtypes(include="object").columns

print("Categorical Columns:")
print(categorical_columns.tolist())

# Plot count plots
for col in categorical_columns:
    plt.figure(figsize=(12,5))

    sns.countplot(
        y=col,
        data=df,
        order=df[col].value_counts().index[:10],
        hue=col,
        palette="viridis",
        legend=False
    )

    plt.title(f"Count Plot of {col}")
    plt.tight_layout()
    plt.show()
```

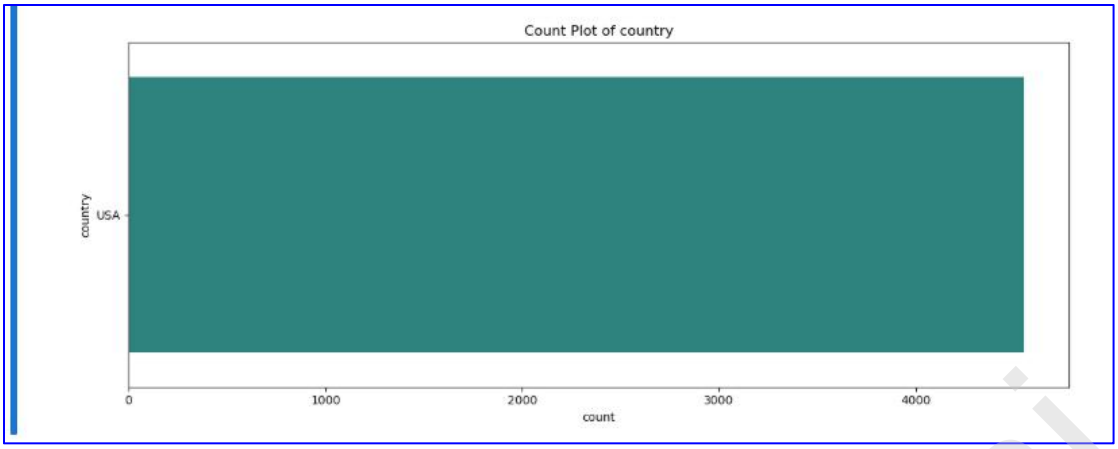
```
Categorical Columns:
['date', 'street', 'city', 'statezip', 'country']
```





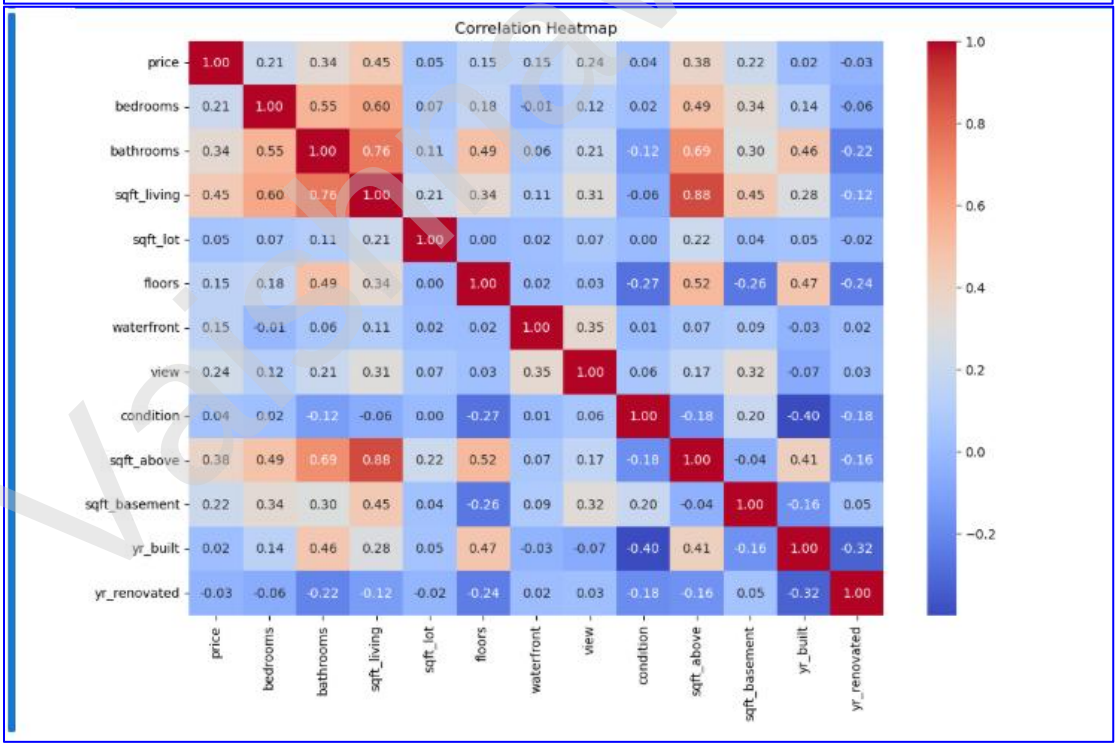






10. Correlation Heatmap

```
[48]: plt.figure(figsize=(12,8))

sns.heatmap(
    df.select_dtypes(include=["int64","float64"]).corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()
```



11. Convert Date into Year and Month

```
[49]: # Convert date into datetime format
df["date"] = pd.to_datetime(df["date"])

# Extract year and month
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month

df.head()
```

[49]:

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	yr_built	yr_ren
0	2014-05-02	313000.0	3.0	1.50	1340	7912	1.5	0	0	3	1340	0	1955	
1	2014-05-02	2384000.0	5.0	2.50	3650	9050	2.0	0	4	5	3370	280	1921	
2	2014-05-02	342000.0	3.0	2.00	1930	11947	1.0	0	0	4	1930	0	1966	
3	2014-05-02	420000.0	3.0	2.25	2000	8030	1.0	0	0	4	1000	1000	1963	
4	2014-05-02	550000.0	4.0	2.50	1940	10500	1.0	0	0	4	1140	800	1976	

12. Drop Unnecessary Columns

```
[50]: # Drop date and street columns

df.drop(["date", "street"], axis=1, inplace=True)

df.head()
```

[50]:

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	yr_built	yr_renovated
0	313000.0	3.0	1.50	1340	7912	1.5	0	0	3	1340	0	1955	2005
1	2384000.0	5.0	2.50	3650	9050	2.0	0	4	5	3370	280	1921	0
2	342000.0	3.0	2.00	1930	11947	1.0	0	0	4	1930	0	1966	0
3	420000.0	3.0	2.25	2000	8030	1.0	0	0	4	1000	1000	1963	0
4	550000.0	4.0	2.50	1940	10500	1.0	0	0	4	1140	800	1976	1992

13. Prepare Independent and Dependent Features

```
[51]: # Independent and Dependent Features

X = df.drop("price", axis=1)

y = df["price"]

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

```
Shape of X: (4551, 17)
Shape of y: (4551,)
```

14. Apply One-Hot Encoding

[52]:
```
# Identify categorical columns
categorical_columns = X.select_dtypes(include="object").columns

print("Categorical Columns:")
print(categorical_columns.tolist())

# Apply One-Hot Encoding
X = pd.get_dummies(
    X,
    columns=categorical_columns,
    drop_first=True
)

print("\nShape After Encoding:", X.shape)

X.head()
```

```
Categorical Columns:
['city', 'statezip', 'country']

Shape After Encoding: (4551, 133)
```

[52]:

	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	...	statezip_WA 98155	statezip_WA 98166	st
0	3.0	1.50	1340	7912	1.5	0	0	3	1340	0	...	False	False	
1	5.0	2.50	3650	9050	2.0	0	4	5	3370	280	...	False	False	
2	3.0	2.00	1930	11947	1.0	0	0	4	1930	0	...	False	False	
3	3.0	2.25	2000	8030	1.0	0	0	4	1000	1000	...	False	False	

15. Apply Feature Scaling

[53]:
```
from sklearn.preprocessing import StandardScaler

# Numerical columns
numerical_columns = [
    "bedrooms",
    "bathrooms",
    "sqft_living",
    "sqft_lot",
    "floors",
    "waterfront",
    "view",
    "condition",
    "sqft_above",
    "sqft_basement",
    "yr_built",
    "yr_renovated",
    "year",
    "month"
]

# Apply Standard Scaling
scaler = StandardScaler()

X[numerical_columns] = scaler.fit_transform(X[numerical_columns])

# Display first 5 rows
X.head()
```

[53]:

	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_basement	...	statezip_WA 98155	statezi
0	-0.436308	-0.843810	-0.828976	-0.192527	-0.022648	-0.08146	-0.306647	-0.665622	-0.564425	-0.671413	...	False	
1	1.774869	0.444408	1.587735	-0.160880	0.905905	-0.08146	4.920139	2.296965	1.811625	-0.065270	...	False	
2	-0.436308	-0.199701	-0.211721	-0.080319	-0.951201	-0.08146	-0.306647	0.815672	0.126151	-0.671413	...	False	
3	-0.436308	0.122354	-0.138487	-0.189245	-0.951201	-0.08146	-0.306647	0.815672	-0.962384	1.493385	...	False	
4	0.669281	0.444408	-0.201259	-0.120558	-0.951201	-0.08146	-0.306647	0.815672	-0.798519	1.060425	...	False	

5 rows × 133 columns

16. Split the Dataset into Training and Testing Sets

[54]:
```
# Split the dataset

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.33,
    random_state=42
)

print("X_train Shape:", X_train.shape)
print("X_test Shape :", X_test.shape)
print("y_train Shape:", y_train.shape)
print("y_test Shape :", y_test.shape)
```

```
X_train Shape: (3049, 133)
X_test Shape : (1502, 133)
y_train Shape: (3049,)
y_test Shape : (1502,)
```

17. Build and Train the Linear Regression Model

[55]:
```
# Create model

model = LinearRegression()

# Train model
model.fit(X_train, y_train)

print("Model Trained Successfully!")

print("\nIntercept:")
print(model.intercept_)

coefficients = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_
})

print("\nModel Coefficients:")
print(coefficients)
```

```
Model Trained Successfully!

Intercept:
379705.4195751214

Model Coefficients:
                  Feature    Coefficient
0                bedrooms  -30152.486829
1               bathrooms   26344.252824
2             sqft_living  112549.800818
3                sqft_lot    3446.158053
4                  floors  -49106.930750
..                    ...            ...
128  statezip_WA 98188 -241991.711201
129  statezip_WA 98198 -363414.626026
130  statezip_WA 98199  447164.758260
131  statezip_WA 98288   19699.444416
132  statezip_WA 98354   10098.600547

[133 rows x 2 columns]
```

18. Make Predictions

[56]:
```
# Predict on test data

y_pred = model.predict(X_test)

comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

comparison.head(10)
```

[56]:

	Actual Price	Predicted Price
0	1225000.0	1.302223e+06
1	496752.0	5.655586e+05
2	612500.0	5.580945e+05
3	265000.0	2.468460e+05
4	615000.0	6.294796e+05
5	432000.0	4.098819e+05
6	305000.0	2.988157e+05
7	405000.0	2.937551e+05
8	349000.0	4.435321e+05
9	839000.0	8.493213e+05

19. Evaluate the Model

[57]:
```
# Evaluation Metrics

r2 = r2_score(y_test, y_pred)

print("R² Score :", round(r2,4))
print("Accuracy :", round(r2*100,2), "%")

if r2 >= 0.90:
    print("\nExcellent Model Performance")
elif r2 >= 0.75:
    print("\nVery Good Model Performance")
elif r2 >= 0.50:
    print("\nGood Model Performance")
else:
    print("\nModel Needs Improvement")
```

```
R² Score : 0.6545
Accuracy : 65.45 %

Good Model Performance
```

20. Save the Trained Model

[60]:
```
# Save model and preprocessing objects

joblib.dump(model, "LR_House_Price.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(X.columns.tolist(), "columns.pkl")

print("Files Saved Successfully!")

print("1. LR_House_Price.pkl")
print("2. scaler.pkl")
print("3. columns.pkl")
```

```
Files Saved Successfully!
1. LR_House_Price.pkl
2. scaler.pkl
3. columns.pkl
```

21. Summary

Summary

- **Dataset Used**
  - House Price Prediction Dataset
- **Data Preprocessing**
  - Removed missing values and duplicate records.
  - Removed invalid price values.
  - Converted the `date` column into `year` and `month`.
  - Dropped the `street` column.
- **Exploratory Data Analysis**
  - Generated statistical summaries.
  - Created histograms, count plots, and a correlation heatmap.
- **Feature Engineering**
  - Applied One-Hot Encoding to categorical features.
  - Applied Standard Scaling to numerical features.
- **Machine Learning**
  - Split the dataset into training and testing sets.
  - Built and trained a Linear Regression model.
  - Predicted house prices using the trained model.
  - Evaluated the model using R^2 Score.
- **Model Saving**
  - Saved the trained model, scaler, and feature columns using Joblib.